Deployment of Machine Learning Model using Fast-API and Docker

---

1. Objective

The objective of this project is to develop a REST API for a machine learning model using Fast-API and containerize it using Docker for easy deployment and scalability.

---

2. Tools & Technologies Used

- Python
- Fast-API
- Uvicorn
- Docker

---

3. Project Description

In this project, a machine learning model is wrapped inside a Fast-API application. The API exposes endpoints that allow users to send input data and receive predictions.

The application is then containerized using Docker, ensuring consistent execution across different environments.

---

4. API Endpoints

GET /

- Description: Checks whether the API is working
- Response:

```
{
  "message": "API is working"
}
```

POST /predict

- Description: Takes input data and returns prediction

Request:

```
{
  "input": 5
}
```

Response:

```
{
  "prediction": 10
}
```

---

5. Implementation Steps

1. Created a Fast-API application (`app.py`)
2. Defined input model using Pydantic
3. Implemented `/predict` endpoint
4. Tested API using Swagger UI
5. Created `requirements.txt`
6. Wrote Dockerfile for containerization
7. Built Docker image
8. Ran Docker container
9. Uploaded project to GitHub

---

6. Docker Commands Used

Build Docker Image:

```
docker build -t ml-api .
```

Run Docker Container:

```
docker run -p 8000:8000 ml-api
```

---

7. Testing the API

Using Swagger UI:

http://localhost:8000/docs

Using cURL:

```
curl -X POST "http://localhost:8000/predict" -H "Content-Type:
application/json" -d "{\"input\": 5}"
```

---

8. Output

The API successfully returns predictions based on input values.

Example:

```
{
  "prediction": 10
}
```

---

9. Conclusion

The project successfully demonstrates how to deploy a machine learning model using FastAPI and Docker. It provides a scalable and portable solution for serving ML models as APIs.

---

10. GitHub Repository

https://github.com/archana196/ml-api-project